EOSC 213: Computational Methods in Geological Engineering

Lecture 21: Automated optimization and parameter estimation using PyTorch

Shadab Ahamed

Department of Earth, Ocean and Atmospheric Sciences,
The University of British Columbia

Thu, Apr 2, 2026

Where we are in the course

In Lecture 20, we introduced parameter estimation for ODE models:

- choose a parameter value,
- solve the ODE numerically,
- compare the prediction to the data,
- compute a loss.

There, we used a **scan over many trial values** of one parameter.

Today's new step

We now ask: **How can we fit parameters automatically, without manually scanning a giant grid?**

What we will do today

By the end of today, you should be able to:

1. set up a **two-parameter** fitting problem,
2. visualize the loss over a 2D parameter space,
3. understand why grid search becomes impractical in higher dimensions,
4. explain what **autograd** does,
5. use a PyTorch optimizer to update parameters automatically.

Main message

Grid search is useful for intuition.
Autograd and optimization are useful for actually solving larger parameter-estimation problems.

From one parameter to two parameters

Last time, we used

$$y' = -\lambda y, \qquad y(0) = 10,$$

and estimated the single parameter λ.

Today we move to a slightly richer model:

$$y' = -ay + b, \qquad y(0) = 10.$$

Now there are **two unknown parameters**:

- a: controls the decay rate,
- b: adds a constant forcing term.

Different choices of (a, b) produce different solution curves.

Why this model is a good next example

The model

$$y' = -ay + b$$

is still simple enough to understand, but already rich enough to show new ideas.

It lets us see that:

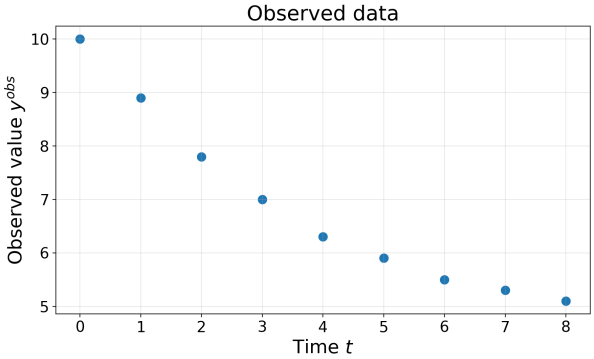
- one of the parameters changes the **rate** of decay,
- the other parameter changes the **long-term level**,
- different parameter pairs can produce somewhat similar curves,
- fitting now means minimizing a function of **two variables**.

Key shift

Instead of a loss curve $L(\lambda)$, we now have a **loss surface** $L(a, b)$.

Observed data for today's example

Assume we measured $y(t)$ at several times and obtained noisy data.



Time t	Observed value y^{obs}
0	10.0
1	8.9
2	7.8
3	7.0
4	6.3
5	5.9
6	5.5
7	5.3
8	5.1

Our goal is to estimate the pair (a, b) that best explains these data.

What does one parameter guess mean now?

A single guess is no longer one number like λ.
Now one guess is a **pair**:

$$(a, b).$$

For each chosen pair (a, b), we do the same workflow:

1. solve the ODE numerically,
2. evaluate the solution at the observation times,
3. compute residuals,
4. compute the loss.

Same structure, bigger search space

The fitting idea did not change.
What changed is the number of parameters we must search over.

Residuals and loss for two parameters

For observation times t_i, the residual is

$$r_i(a, b) = y_{a,b}(t_i) - y_i^{obs}.$$

The least-squares loss is

$$L(a, b) = \frac{1}{N} \sum_{i=1}^{N} r_i(a, b)^2 = \frac{1}{N} \sum_{i=1}^{N} \left(y_{a,b}(t_i) - y_i^{obs}\right)^2.$$

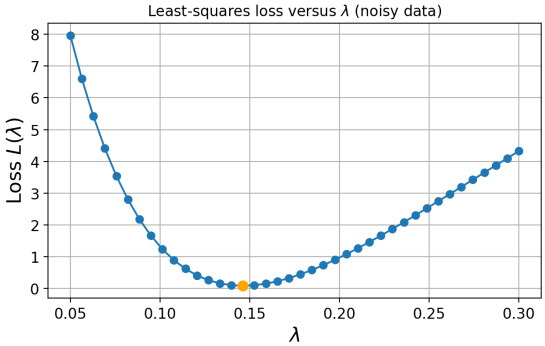
So now:

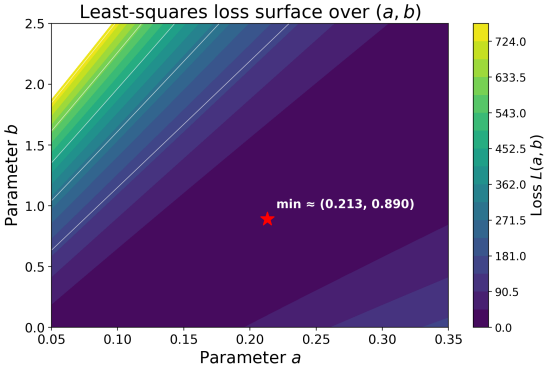
- low loss means the pair (a, b) fits well,
- high loss means the pair (a, b) fits poorly,
- the best-fit parameters are at the **minimum** of the surface $L(a, b)$.

From a loss curve to a loss surface

In Lecture 20: $L(\lambda)$ was a function of one variable.

In Lecture 21 (today): $L(a, b)$ is a function of two variables.





The minimum is no longer just the lowest point on a curve.

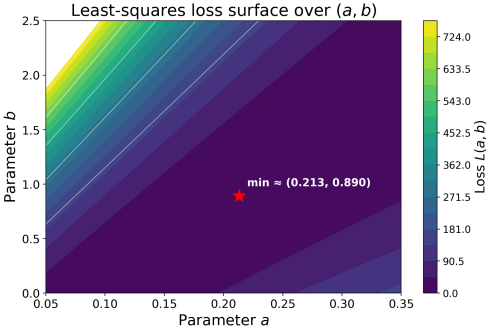
It is the lowest point on a **surface** or heatmap.

Visualizing the loss on a 2D plot

Each point in the (a, b)-plane corresponds to one parameter pair.

For each point:

- solve the ODE,
- compute the loss,
- color the point by how large the loss is.

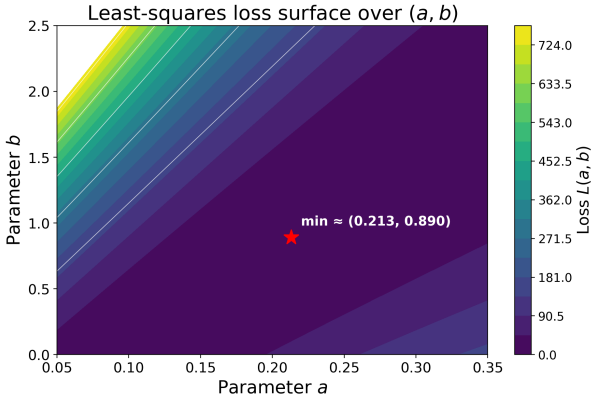


Interpretation:

- bright/high region $\Rightarrow$ poor fit,
- dark/low region $\Rightarrow$ good fit,
- darkest point $\Rightarrow$ best-fit pair.

Finding the minimum on the 2D loss plot

Once the loss values are computed on the grid, we can identify the minimum.



The marked point shows the parameter pair

$$(a_\star, b_\star)$$

with the smallest grid-searched loss.

This is very useful because it makes the geometry of the optimization problem visible.

But there is a limit

This is easy to visualize only when there are one or two parameters.

A 2D grid search idea

In 2D grid search, we:

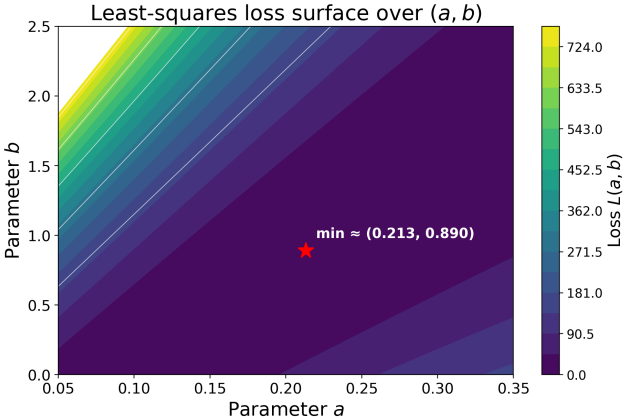
1. choose many trial values of a,
2. choose many trial values of b,
3. test every pair (a, b),
4. compute the loss for each pair,
5. find the pair with the smallest loss.

For example:

- search over 50 values for a,
- search over 60 values for b

means

$$50 \times 60 = \textbf{3000 ODE solves.}$$



This is inefficient

A very inefficient way to search the 2D space.

Why grid search does not scale well

Suppose we have p unknown parameters in a problem and we test **100** trial values per parameter.

Then the total number of combinations to check is:

$$\underbrace{100 \times 100 \times \cdots \times 100}_{p \text{ times}} = 100^p$$

Examples:

- $p = 1$: 100 combinations,
- $p = 2$: $100^2 = 10{,}000$ combinations,
- $p = 3$: $100^3 = 1{,}000{,}000$ combinations,
- $p = 5$: $100^5 = 10^{10}$ combinations.

Main issue

Grid search grows exponentially with the number of parameters.
This is one reason we need optimization methods.

What do we really want instead?

Instead of testing every possible parameter combination, we want a method that:

- starts from an initial guess,
- knows which direction reduces the loss,
- updates the parameters automatically,
- repeats until the loss becomes small.

That is the basic idea of **gradient-based optimization**.

Central question

How do we compute the derivatives of the loss with respect to the parameters?

From grid search to a downhill method

In Lecture 20, we studied a one-parameter loss $L(\lambda)$.

We found the best λ by:

- trying many values of λ,
- computing the loss each time,
- choosing the one with the smallest loss.

This is a **grid search** approach.

But instead of checking many points one by one, can we use the **shape** of the loss function to move toward the minimum automatically?

Gradient descent idea in one dimension

Suppose the loss depends on only one parameter:

$$L(\lambda).$$

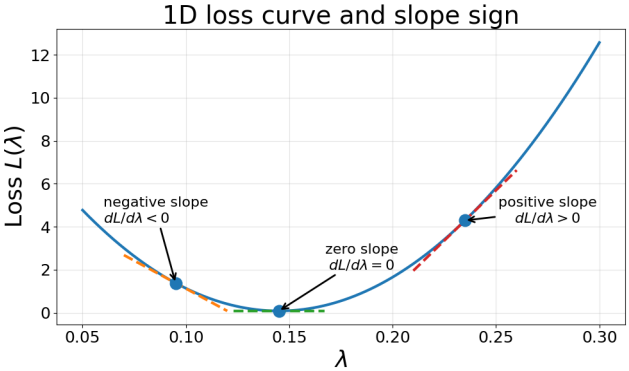
Its derivative

$$\frac{dL}{d\lambda}$$

tells us the local slope of the loss curve.

- If $\frac{dL}{d\lambda} > 0$, the loss increases as λ increases.
- If $\frac{dL}{d\lambda} < 0$, the loss decreases as λ increases.

So to reduce the loss, we move in the direction opposite to the slope.



One-dimensional gradient descent update

A basic gradient descent update rule is

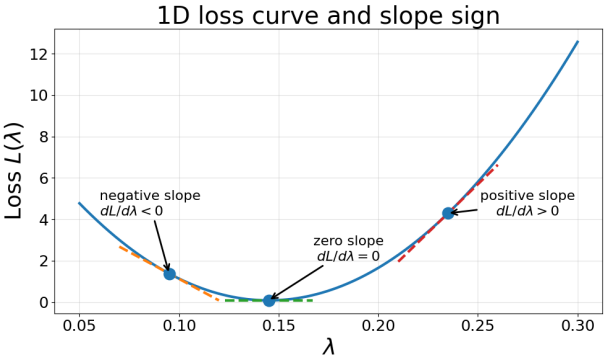
$$\boxed{\lambda_{k+1} = \lambda_k - \eta \frac{dL}{d\lambda}(\lambda_k)}$$

where $\eta > 0$ is the learning rate.

Interpretation:

- λ_k: current guess,
- $\frac{dL}{d\lambda}(\lambda_k)$: slope at the current guess,
- η: step size (aka learning rate).

If the slope is **positive**, **we move left**.
If the slope is **negative**, **we move right**.



Summary

This update tries to move **downhill** on the loss curve.

From one parameter to two parameters

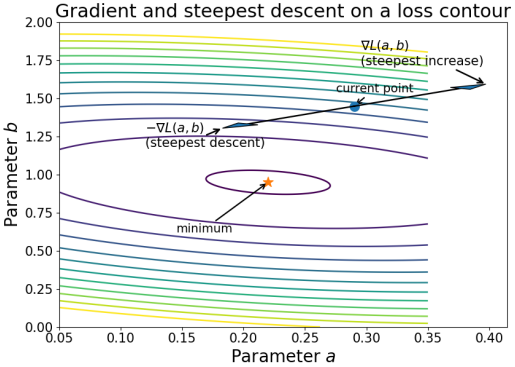
Now suppose the loss depends on two parameters:

$$L(a, b).$$

Instead of a loss **curve**, we now have a loss **surface**.
In one dimension, the derivative gave the local slope.
In two dimensions, the analogue is the **gradient**:

$$\boxed{\nabla L(a, b) = \left(\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b}\right)}$$

This tells us how the loss changes when we change a and b.



Gradient descent idea in two dimensions

For the two-parameter loss $L(a, b)$, the gradient is

$$\nabla L(a, b) = \left(\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b} \right).$$

This gradient points in the direction of **steepest increase** of the loss.

So to decrease the loss, we move in the opposite direction.

A basic gradient descent update is:

$$a_{k+1} = a_k - \eta \frac{\partial L}{\partial a}, \qquad b_{k+1} = b_k - \eta \frac{\partial L}{\partial b}.$$

This is the two-dimensional version of the one-dimensional update.

Gradient descent in higher dimensions

If a model has many parameters

$$\theta = (\theta_1, \theta_2, \ldots, \theta_m),$$

then the loss is a function

$$L(\theta_1, \theta_2, \ldots, \theta_m).$$

The gradient is

$$\nabla L(\theta) = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \ldots, \frac{\partial L}{\partial \theta_m} \right).$$

Gradient descent becomes

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla L(\theta^{(k)}).$$

This is why gradient-based optimization is so useful: it still works even when the number of parameters is far too large for grid search.

What is autograd?

Autograd is PyTorch's automatic differentiation system.

It keeps track of operations used to compute a quantity such as the loss, and then applies the chain rule automatically.

So if the loss depends on

$$\text{parameters} \to \text{numerical solution} \to \text{residuals} \to \text{loss},$$

then PyTorch can compute the derivatives of the loss with respect to the parameters.

Big picture

You write the forward computation.
PyTorch computes the derivatives for you.

Why this is powerful for our ODE fitting problem

In parameter estimation, the loss depends on the parameters **through the numerical solver**.

For our example:

$$(a, b) \longrightarrow \text{Forward Euler solution} \longrightarrow y_{a,b}(t_i) \longrightarrow L(a, b).$$

Autograd can differentiate through this whole computation, as long as we implement it using differentiable PyTorch operations.

Interpretation

The solver is no longer just producing numbers.
It is part of a **differentiable computation graph**.

PyTorch setup for the observed data

```
import torch

# observed times and data
t_obs = torch.tensor([0., 1., 2., 3., 4., 5., 6., 7., 8.])
y_obs = torch.tensor([10.0, 8.9, 7.8, 7.0, 6.3,
                      5.9, 5.5, 5.3, 5.1])

y0 = torch.tensor(10.0)
```

These are the data we want the model to fit.

Forward Euler solver with two parameters

```
def forward_euler_model(a, b, h=0.1, t_final=8.0):
    nsteps = int(t_final / h)
    y = y0

    values = [y]
    for _ in range(nsteps):
        y = y + h * (-a * y + b)
        values.append(y)

    return torch.stack(values)
```

Notice that the parameters a and b appear inside the time-stepping loop.

If a and b require gradients, PyTorch will track all of these operations.

Loss function for a chosen pair (a, b)

```
def loss_fn(a, b):
    y_model = forward_euler_model(a, b)

    # with h = 0.1 and t_obs = 0,1,...,8,
    # matching indices are 0,10,20,...,80
    idx = torch.tensor([0,10,20,30,40,50,60,70,80])
    y_pred = y_model[idx]

    return torch.mean((y_pred - y_obs)**2)
```

This returns the least-squares mismatch between prediction and data.

Grid search over two parameters

```
a_values = torch.linspace(0.1, 0.8, 80)
b_values = torch.linspace(0.0, 4.0, 80)

loss_grid = torch.zeros(len(a_values), len(b_values))

for i, a in enumerate(a_values):
    for j, b in enumerate(b_values):
        loss_grid[i, j] = loss_fn(a, b)
```

This produces a table of loss values that can be displayed as a heatmap.

It is excellent for visualization, but expensive when the number of parameters increases.

Making the parameters learnable

```
a = torch.tensor(0.30, requires_grad=True)
b = torch.tensor(1.00, requires_grad=True)
```

Setting `requires_grad=True` tells PyTorch:

- track operations involving these tensors,
- compute derivatives of the final loss with respect to them.

After we compute the loss, we can call

```
loss.backward()
```

and PyTorch will fill in

```
a.grad    b.grad
```

with the corresponding derivatives.

A single autograd step

```
a = torch.tensor(0.30, requires_grad=True)
b = torch.tensor(1.00, requires_grad=True)

loss = loss_fn(a, b)
loss.backward()

print(a.grad)
print(b.grad)
```

These gradients tell us the local slope of the loss surface.

Interpretation:

- if `a.grad` is positive, decreasing a may reduce the loss,
- if `a.grad` is negative, increasing a may reduce the loss,
- similarly for b.

Using a PyTorch optimizer

```
a = torch.tensor(0.30, requires_grad=True)
b = torch.tensor(1.00, requires_grad=True)

optimizer = torch.optim.SGD([a, b], lr=0.05)
```

The optimizer will update the parameters for us.

Basic loop:

1. compute loss,
2. compute gradients,
3. update parameters,
4. repeat.

Optimization loop in PyTorch

```
for k in range(200):
    optimizer.zero_grad()
    loss = loss_fn(a, b)
    loss.backward()
    optimizer.step()
```

What each line does:

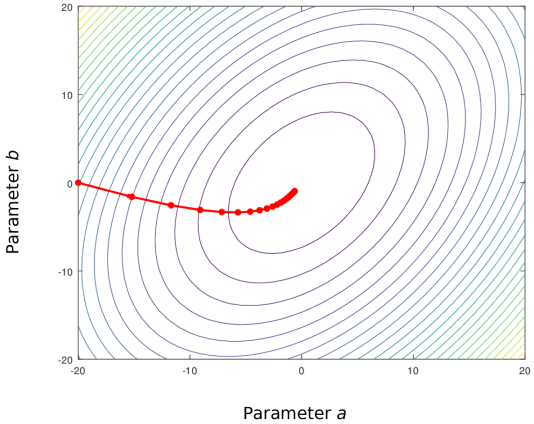
- `zero_grad()` clears old gradients,
- `loss_fn(a,b)` computes the current loss,
- `backward()` computes new gradients,
- `step()` updates a and b.

How to interpret the optimization path

Starting from an initial guess, the optimizer generates a sequence

$$(a_0, b_0),\ (a_1, b_1),\ (a_2, b_2),\ \ldots$$

that ideally moves toward a minimum of the loss.



Each step tries to move downhill on the loss surface.

This gives a geometric picture of what optimization is doing.

Why this beats grid search in larger problems

Grid search tries **many combinations blindly**.

Gradient-based optimization tries to use **local derivative information** to move intelligently toward better parameters.

So in many practical problems, optimization can be much more efficient than evaluating a giant grid.

But not magic

Optimization still depends on:

- the starting guess,
- the learning rate,
- the shape of the loss surface,
- the quality of the numerical solver.

A few practical remarks

- The parameters should be initialized to reasonable values.
- A learning rate that is too large may cause unstable updates.
- A learning rate that is too small may make optimization very slow.
- Noise in the data can affect the fitted parameters.
- Numerical error from the ODE solver can also affect the fit.

Connection to the course

The optimizer is not replacing numerical analysis.
It relies on the numerical solver being implemented carefully.

What to notice from this lecture

1. With two parameters, the loss becomes a surface instead of a curve.
2. A 2D grid search is possible and very instructive.
3. Grid search becomes impractical as the number of parameters grows.
4. Autograd computes derivatives of the loss automatically.
5. Optimizers use those derivatives to update parameters efficiently.

Summary of Lecture 21

Today we moved from **manual scanning** to **automatic optimization**.

Core ideas:

- a two-parameter model leads to a 2D loss surface,
- grid search is useful for visualization but scales poorly,
- PyTorch autograd computes derivatives through the numerical solver,
- gradient-based optimizers update parameters automatically.

Next lecture

We will use these ideas in a richer nonlinear example and discuss practical fitting issues such as initialization, noise, and optimizer behavior.